

Tacit: Confidential Assets and DeFi on Bitcoin, Bridged by Zero-Knowledge Reflection

z0r0z
<https://tacit.finance>

Version 1 2026

Abstract

Bitcoin secures value without a trusted party, yet every way to issue assets on it, hide their amounts or trade them has brought one back: a proof courier, a federation, a rollup operator or a custodian. Tacit is a confidential asset layer for Bitcoin that needs none of them.

Tacit assets live on Bitcoin in Taproot envelopes, with amounts hidden in Pedersen commitments, and any indexer derives the same state from the chain alone. The same notes extend into an immutable pool on Ethereum that accepts private transactions only with an SP1 zero-knowledge proof. Each chain proves its state to the other, Bitcoin by proof-of-work and Ethereum by its light client, so value and calls cross between them with no signer, multisig or custodian.

In the pool, users send, swap, borrow and earn with amounts hidden, and never need ETH for gas. cBTC brings in real BTC from self-custody locks, and every balance recovers from a single key. The core is immutable, so nobody can change the rules under a user's funds, yet the protocol keeps improving, pool by pool, toward a V2 built on Bitcoin covenants.

1 Introduction

A useful asset layer on Bitcoin needs five properties:

1. its state is public data anyone can check;
2. amounts are hidden;
3. value can move to where computation is cheap without trusting a bridge;
4. BTC itself can back a fungible asset;
5. a user can recover everything from a key.

Existing designs each give up one:

Design	What it gives up
Ordinals, Runes	Amounts are public.
RGB, Taproot Assets	Validity moves off-chain, so losing a proof means losing the balance.
Liquid	Relies on a federation.
Rollups and Bitcoin L2s	Rely on an operator set.
Wrapped BTC	Relies on a custodian or a threshold group.

Tacit delivers all five by combining existing systems, with no new consensus:

Part	What it offers
Bitcoin metaprotocol (§2, §3)	Issue, send and trade Bitcoin assets with hidden amounts.
Confidential pool (§4, §6)	Private sends, swaps, loans and farms on Ethereum, gas-free.
Reflection (§5)	Notes and calls move between chains by proof, with no signer.
cBTC (§6)	Real BTC in self-custody locks, private on both chains.
Lineage (§8, §11)	An immutable core that improves by succession, toward covenants.

One property Bitcoin cannot yet enforce on its own: custody of the BTC behind cBTC. Tacit secures it economically until covenants make it absolute (§11).

2 A metaprotocol on Bitcoin

A Tacit op is a commit/reveal pair. The commit pays to a Taproot output whose only script leaf is:

$$\langle P \rangle \text{ OP_CHECKSIG OP_FALSE OP_IF "TACIT" 0x01 } \langle \text{payload} \rangle \text{ OP_ENDIF}$$

The reveal spends the commit by script path, so the payload rides in witness data. Bitcoin nodes ignore it. Indexers read it: a Tacit UTXO is valid iff the op that created it passes its rules and each of its asset inputs is itself valid.

The rules are deterministic, so two indexers that see the same bytes reach the same verdict. There is no quorum, vote or leader, the same trust model as Ordinals and Runes. Users trust only the published specification and its open implementations. The Bitcoin reflection guest (§5) checks the rules a second time, independently, under a verifying key that cannot change.

Ops cover:

- **Issuance:** confidential supply, or fair-launch mints against a public cap. TAC, the native asset, has a fixed supply of 21 million.
- **Transfers:** hidden amounts, with optional stealth addresses.
- **Trades:** atomic trades against BTC, bids that fill while the buyer is offline, and a native AMM with swaps, routes, uniform-price batches and LP farms.
- **cBTC:** locks and redeems of real BTC.
- **Cross-chain:** bridge burns, re-mints of notes returning from Ethereum, and value-free calls in both directions.

An unknown opcode creates no state, so the protocol grows by adding opcodes.

3 Confidential value

Every amount is a Pedersen commitment on secp256k1:

$$C = v \cdot H + r \cdot G,$$

where G is Bitcoin's generator and H is a nothing-up-my-sleeve point hashed from the tag `tacit-generator-H-v1`. Commitments hide the value perfectly and add homomorphically. A

transaction balances iff its excess

$$E = \sum C_{\text{out}} + b \cdot H - \sum C_{\text{in}}$$

has no H component, where b is any public burn. Tacit proves this with a BIP-340 signature under E , a Mimblewimble-style kernel. Only someone who knows every blinding can sign, and nobody can sign for an excess that hides value.

Range proofs bound every output to $[0, 2^{64})$, aggregated over up to eight outputs, using Bulletproofs+ (classic Bulletproofs remain valid). The recipient's blinding and the key that encrypts its amount come from an ECDH secret between sender and recipient, so a recipient finds and opens every payment from its own key and the chain, with no share link and no sync server.

The Ethereum pool uses the very same commitment. One note format on both chains lets value cross between them unchanged.

4 The confidential pool

The pool is one immutable contract with no owner and no pause. It holds three depth-32 keccak Merkle trees, for notes, locks and CDP positions, each with its own spent set. It escrows ETH and outside tokens, and holds the reflected state of Bitcoin.

Notes. A note is $(asset, C, owner)$, stored as a keccak leaf. A note that came from Bitcoin commits instead to its Bitcoin key and to the pool it is bound to. Spending a note reveals its nullifier. An owned note's nullifier includes the owner's secret key, so no observer can tell which leaf a spend consumes. Bearer and Bitcoin-homed notes use a nullifier derived from the leaf alone.

Entry and exit. `wrap` escrows ETH or any ERC-20 as a deposit that only its depositor's proof can claim, and one transaction can wrap and immediately transfer, swap, add liquidity or borrow. TAC, cBTC and cUSD have ERC-20s that only the pool mints: unwrapping mints them, and wrapping burns them. A note can also exit partway, as a public payout plus hidden change.

Settlement. Every change to notes, locks and positions is a batch of up to 256 ops, proven by one SP1 program. For each op, the program checks:

- that each input is in a known root, without revealing which leaf;
- the range proofs;
- the kernel or opening proofs;
- the op's own rules.

It then commits the public effects: nullifiers, new leaves, payouts, fees, swaps, liquidity changes, bridge claims and memos. The contract verifies the proof against a pinned verifying key and applies exactly those effects.

A spend may prove membership against any root the tree has ever had. Proofs never go stale, and an owned note hides among every owned note of its asset. Costs never grow with use:

- an insert is 32 hashes at any size;
- nullifier and root sets are constant-time maps, never iterated;
- the guest reaches history only through fixed-depth proofs.

As a floor against inflation that holds independently of the proof, the contract never records more spent notes than it has created. Every proof is bound to

$$CHAIN_BINDING = \text{keccak}(chainid || address),$$

so it is invalid in any other deployment.

5 Reflection: a bridge without signers

Bitcoin to Ethereum. Anyone may extend `BitcoinLightRelay`, a header relay that checks full proof-of-work and follows the heaviest chain. The reflection guest proves a run of blocks on top of it. For each block it:

1. re-hashes every transaction to the header’s Merkle root;
2. marks spent every input that spends a reflected note;
3. re-verifies and folds each Tacit envelope it understands, and skips the rest;
4. extends a digest chain over the result.

The pool accepts this proof only if it extends the last one, its tip is buried at least 24 blocks under the relay tip, and its counters match the pool’s own. The pool then holds Bitcoin’s Tacit note roots, spent set, bridge burns, cBTC locks and pending calls.

Value enters the pool from Bitcoin in two ways:

- **Bridge burn.** The note is destroyed on Bitcoin under a burn id that names the target pool, and that pool mints it exactly once.
- **Fast lane.** A Bitcoin note bound to this pool is spent directly on Ethereum. Its proof shows the note is absent from Bitcoin’s reflected spent set and carries a signature from its Bitcoin key. Reflection later retires the note on Bitcoin.

Ethereum to Bitcoin. A second guest, built on the `sp1-helios` sync-committee light client, proves the pool’s finalized storage: cross-outs, fast-lane spends and outbox messages. The Bitcoin reflection guest verifies this proof recursively, then:

- re-mints on Bitcoin the notes that crossed out of the pool;
- delivers outbox messages;
- retires fast-lane notes.

Cross-chain calls. Beside value, reflection carries authenticated calls in both directions, so each chain can act on the other. Neither channel carries value: a call commits no amount, so it can authorize no mint and no transfer.

- **Bitcoin to Ethereum.** A `T_BTC_CALL` envelope is a Bitcoin key’s Schnorr-signed authorization of one Ethereum call: a target contract and its calldata. The reflection guest checks the signature, and the pool records the call in its inbox of pending calls. `BtcCallExecutor`, a separate contract with no privileges, fires it exactly once, and anyone may trigger it. The target sees which Bitcoin key signed and gates on it, so a Bitcoin key can own and operate an Ethereum contract, such as a smart account, a vault or a governance module, with no Ethereum key at all.
- **Ethereum to Bitcoin.** Any Ethereum contract can post a message to `EthCallOutbox` under a handler namespace. The light-client proof carries it to Bitcoin, where a `T_ETH_CALL` envelope delivers it into Bitcoin’s inbox of honored messages, stamped with the sending contract’s address.

A timelock, a DAO or an oracle on Ethereum becomes an authority that Bitcoin-side logic can follow, with no new keys.

These channels let Bitcoin keys run Ethereum accounts and governance, let TAC holders on Bitcoin act on Ethereum, and let Ethereum contracts push attestations, parameters and data to Bitcoin applications. Ethereum-to-Bitcoin delivery is at least once: a sender confirms its message landed and resends if not, which suits idempotent actions.

The bridge rests on Bitcoin proof-of-work, the Ethereum sync committee and SP1, with no multisig, attestor set or IOU. It needs only someone to run the provers. Anyone may, and Ethereum-side notes stay spendable even if nobody does. A full round trip has settled on mainnet: a note left the pool, was re-minted on Bitcoin, was burned there, and was minted back, proven in both directions.

6 Confidential DeFi

Swaps. Pools are constant-product curves with a fee tier and an optional protocol fee, and their reserves are public. `TacitPublicAmm` trades against the same reserves in the clear, so public and private liquidity share one curve.

- `OP_SWAP_ROUTE` swaps a hidden note through up to four pools, with an end-to-end minimum output.
- `OP_SWAP` clears many hidden orders at one uniform price. Everyone in a batch pays the same price, so nobody can sandwich anyone, and the batch reveals only its net change to reserves.

Prover-blind swaps. Whoever proves an ordinary swap sees its amounts. `OP_SWAP_BLIND` hides them from the SP1 prover as well. Each trader commits to its input on BabyJubJub, the curve native to BN254, and a 169-byte cross-curve proof ties that commitment to its secp256k1 note. One Groth16 proof of the `amm_swap_batch` circuit shows, over hidden amounts, that the price, every fill and every minimum output are correct. Only the batcher sees the amounts; the SP1 prover and the chain never do.

Uniform-price clearing over hidden amounts is cheap as a circuit and costly as a zkVM program, so this one step uses a circuit. Its key comes from a trusted-setup ceremony (Hermez Phase 1, then a Phase 2 of 5,018 contributions sealed by a Bitcoin-block beacon) and is compiled into both SP1 guests, so their own verifying keys fix it. The same circuit clears `T_SWAP_BATCH` on Bitcoin.

Trades without a pool. `OP_OTC` swaps two parties' notes directly. `OP_BID` is a limit order with a pre-signed grid of partial fills, so the buyer need not be online. Adaptor locks release a note when a Schnorr signature is revealed, which gives atomic swaps with the other chain.

Stealth payments. A payer locks a note under a one-time key derived from the recipient's public key. The recipient claims it with a signature, and the payer can take it back after a deadline. A Bitcoin bridge burn can mint straight into such a lock.

cBTC. A user locks BTC on Bitcoin with a `T_CBTC_LOCK` envelope that names the note it will mint. Reflection records the lock's value, and the pool mints exactly that much cBTC, once. The locker keeps the lock's key. An escrow in wstETH of at least $1.5\times$ the lock's value, which anyone may fund, makes spending the lock a loss: a minted lock spent without a matching redeem shows up in reflection and anyone can slash the escrow, while a proper redeem returns it. The amount minted never depends on an oracle; a price feed only sizes the escrow. Custody is economic until covenants make it absolute (§11).

cBTC is a Tacit asset on both chains. It crosses out of the pool and is re-minted on Bitcoin by light-client proof (§5), where it moves with hidden amounts like any other Tacit note.

cUSD. A CDP locks cBTC and mints cUSD. The borrower is hidden; the position's amounts are public, because the controller prices them against a BTC/USD feed. Minting needs 150% collateral, and below 130% anyone can liquidate by repaying the debt and taking the collateral. A stability fee and a savings rate are built in. Policy lives in a governed **CollateralEngine** that the pool only asks to accept or reject, with on-chain caps on its ratios and fee, and a cooling-off period before a feed change or a stricter liquidation ratio takes effect.

Farms. LP shares bond into a receipt note that earns rewards. A farm controller pays harvests from a treasury the pool escrows for it and returns the shares on unbonding. Its governor can reweight pools within capped, delayed bounds, but can never cut a running reward stream short or lower its total rate.

7 Gasless, self-sovereign, recoverable

Gasless. Any op may be relayed. The relayer's fee is part of what the proof commits to, bound by the kernel, the opening proof or the note's spend signature. The relayer pays gas and collects exactly that fee. It cannot redirect a payout, raise the fee or submit after the user's deadline. Once value is in the pool, a user never needs ETH. Entering from Ethereum takes one transaction, which can also tip a relayer to settle the deposit. A Bitcoin-only user needs no ETH at all: it enters by bridge burn or the fast lane paying only BTC, acts through relayers, and exits to Bitcoin paying only BTC or to any Ethereum address. Minting cBTC is the one exception, because its escrow is posted on Ethereum, by the locker or any backer.

Self-sovereign. The relayer is optional. Anyone can call **settle**, proofs can be made locally or on an open prover network, and all three guest programs rebuild byte for byte from source. A user can prove locally, settle directly, and involve no one.

Recoverable. A wallet rebuilds every balance and position from its key and public chain data. Bitcoin payments are found by trial derivation (§3). Pool notes carry memos encrypted to their owner and committed in the proof that created them, and positions are read back from the pool's events. There is no backup file to lose and no server to ask.

8 An immutable core that keeps improving

The pool, its guests and their verifying keys cannot be upgraded, so nobody can change the rules under a note that already exists. The protocol still improves, in two ways that never touch the core.

At the edges. New Bitcoin opcodes are additive, so indexers adopt them without disturbing old ones. Anyone can bring an ERC-20 into the pool. New debt assets and reward programs plug in as controllers that the pool only asks to accept or reject. Clients, relayers and provers improve freely, because they only produce proofs the fixed guests already check. V1 deepens as its market grows, with no migration.

By succession. The pool live today is the root of the V1 lineage. Each pool can deploy its own successor through **createNextGen**, its one privileged call, held by the ops multisig. The call runs once, from the pool's own address, and a successor accepts only the predecessor that deployed it.

The steward chooses the successor's code and nothing else: it cannot touch escrow, freeze an exit or redirect a payout. Succession carries state forward:

- **The Bitcoin lane continues without a gap.** The successor's first reflection proof builds on the predecessor's last attested state, read live from its contract. Value on Bitcoin never has to move.
- **Assets keep their identity.** Asset ids are shared across chains and pools, so TAC, tETH and every Bitcoin asset are the same asset in each pool.
- **The old pool keeps its promises.** It takes no new value, but every exit, every release of value already committed, and its reflection stay open.
- **Users move on their own schedule,** by exiting one pool and entering the next. Nothing moves their value for them, and a pool deployed by anyone else cannot touch the lineage.

9 Privacy

	Hidden	Public
Bitcoin	amounts; stealth recipients; trade sizes in T_SWAP_BATCH	addresses, the transaction graph, asset ids, burn amounts, AMM reserves
Pool	amounts; which owned note a spend consumes	the deposit and withdrawal boundary, AMM reserves and their changes, CDP amounts
Relay	your key, seed and blindings	the witness of the ops it proves

An owned note hides among every owned note of its asset, not one fixed denomination. Proving locally keeps a witness on one device, and OP_SWAP_BLIND keeps trade sizes from the prover too.

10 Trust and limits

Relied on:

- Bitcoin and Ethereum consensus;
- the soundness of SP1 and Groth16;
- the spl-helios sync committee;
- for the swap circuit only, at least one honest ceremony contributor;
- the CollateralEngine price feeds: BTC/USD for cUSD, and BTC per wstETH for cBTC escrow;
- the ops multisig (2-of-4, with a one-hour delay unless all four sign), within the on-chain bounds of the collateral and farm controllers;
- for a walk-away bid on Bitcoin only, a watchtower holding a capped, dedicated key over the sats the buyer commits.

Not relied on: relayers, the hosted API, gateways and explorers. The reference indexer reads Bitcoin from public endpoints, and any other source gives the same state.

Limits:

- Reflection halts, rather than rewrites, on a Bitcoin reorg deeper than its confirmation depth.
- Anyone can submit a relayed proof first and collect its fee. The user's outcome is the same, because the proof works for any relayer.
- cBTC custody is economic until covenants.

- Bitcoin transfers carry a large witness, so Tacit on Bitcoin suits settlement, and the pool is where frequent activity belongs.

11 Covenants, V2 and the road ahead

Bitcoin cannot yet restrict where an output may be spent. Once it can, through CTV, OP_CAT, OP_CHECKSIGFROMSTACK, OP_VAULT or an equivalent, three constructions become fully trustless. Tacit reserves room for each:

- **Covenant cBTC.** A lock whose sats can leave only through a redemption. Pool op 5, OP_COVENANT_MINT, is held for its mint, which needs no escrow.
- **On-chain bid escrow.** A buyer's pre-signed input bound to its bid on-chain, replacing the watchtower.
- **Fractional slots.** Reserved opcodes that split a BTC slot into fungible shares and recombine them, with no shared vault.

V2. V1 is built to hand off to this covenant generation. V2's Bitcoin opcodes arrive additively, and its settle logic uses op codes V1 keeps free. V2 deploys through V1's own `createNextGen` and inherits the Bitcoin lane (§8). cBTC then moves from economic to enforced custody one lock at a time: a locker redeems a V1 lock, which frees its escrow, and re-locks the sats in a covenant vault that mints in V2 with no escrow at all. Every V1 note keeps its exits throughout.

A shielded pool on Bitcoin. Bitcoin opcodes 0x6C and 0x6D carry a shielded pool that lives on Bitcoin alone, described in the companion paper *Secret Sats*. It holds Tacit assets, not BTC, so it adds no custody. Notes shield in and exit as ordinary Tacit notes. Each spend proves membership, nullifiers, its owner's signature and conservation in one SP1 proof, so sender and receiver are unlinked and amounts stay hidden. Indexers verify each proof locally with no Ethereum dependency, and relayers post spends, so a sender needs no Bitcoin wallet. cBTC is the pool's path to BTC: locked sats mint cBTC in the Ethereum pool, cBTC crosses back to Bitcoin by light-client proof, and there it shields. Sats also move in and out by atomic swap, and by covenant once Bitcoin allows one.

Cryptography evolves the same way. A better range proof is added alongside the old one, as Bulletproofs+ was, and proof systems verifiable in Bitcoin script could move parts of indexer validation into consensus.

12 Conclusion

Tacit is an asset layer on Bitcoin with no bridge signer and no custodian:

- a deterministic metaprotocol with hidden amounts;
- an immutable, proof-gated pool that carries the same notes into confidential DeFi;
- a bridge made of proofs in both directions, for value and for calls;
- cBTC, which carries real BTC into that DeFi and back to Bitcoin;
- a lineage of immutable pools that improves as its market grows and hands off to V2 when Bitcoin gains covenants.

Where cryptography can enforce a property, it does. Where Bitcoin cannot yet enforce one, the gap is stated, bounded, and reserved for a covenant to close.

References

1. Rodarmor, C. *Ordinals* (2023); *Runes* (2024).
2. Pedersen, T.P. *Non-interactive and information-theoretic secure verifiable secret sharing*. CRYPTO 1991.
3. Jedusor, T.E. *Mimblewimble* (2016); Poelstra, A. *Mimblewimble* (2016).
4. Bünz, B. et al. *Bulletproofs*. IEEE S&P 2018. Chung, H. et al. *Bulletproofs+*. 2020.
5. Wuille, P.; Nick, J.; Ruffing, T.; Towns, A. *BIP-340*, *BIP-341*. 2020.
6. Groth, J. *On the size of pairing-based non-interactive arguments*. EUROCRYPT 2016.
7. Succinct. *SP1 zkVM*; *sp1-helios*.
8. Pertsev, A.; Semenov, R.; Storm, R. *Tornado Cash* (2019).
9. Walras, L. *Éléments d'économie politique pure* (1874); Gnosis Protocol (2019); Penumbra ZSwap (2023).
10. Rubin, J. *BIP-119 (CTV)*; *OP_CAT* and *OP_CHECKSIGFROMSTACK* proposals.
11. Tacit. *Secret Sats* (2026), companion paper.